# HojaTrabajo_IntroduccionPython

April 12, 2026

## 1 Hoja de Trabajo — Introducción a Python

**Nombre:** Joselin Liseth Tzunux Coxaj

**Carnet:** 202606884

**Fecha:** 12/04/2026

---

Instrucciones generales: - Completa cada celda de código donde se indica `# TU CÓDIGO AQUÍ`. - No borres las instrucciones ni los comentarios de guía. - Ejecuta cada celda para verificar que tu solución es correcta. - Puedes consultar el script `introduccion_python.py` como referencia.

---

### 1.1 1. Hola Mundo y `print()`

La función `print()` muestra texto en pantalla. Puede recibir varios argumentos separados por comas y tiene el parámetro `end` para controlar cómo termina la línea.

[15]:
```
# Ejercicio 1a
# Imprime tu nombre y carnet en una sola línea usando print() con dos argumentos.
# Ejemplo de salida esperada:  Nombre: Ana  Carnet: 202300001

print("Nombre: Joselin", end="   ")
print("Carnet: 202606884")
```

```
Nombre: Joselin   Carnet: 202606884
```

[13]:
```
# Ejercicio 1b
# Usa end="" para imprimir las palabras "Hola", "desde" y "Python"
# en una sola línea, cada una en su propia llamada a print().
# Salida esperada:  Hola desde Python

print("Hola", end=" ")
print("desde", end=" ")
print("Python")
```

```
Hola desde Python
```

---

## 1.2 2. Variables y tipos de datos

Python es de **tipado dinámico**: no necesitas declarar el tipo, se asigna automáticamente.
Tipos básicos: `int`, `float`, `str`, `bool`, `NoneType`.

[20]:
```
# Ejercicio 2a
# Crea variables con los siguientes datos sobre ti:
#   - nombre     (str)   con tu nombre completo
#   - edad       (int)   con tu edad
#   - promedio   (float) con un promedio de notas inventado
#   - activo     (bool)  True si eres estudiante activo
# Luego imprime cada variable junto con su tipo usando type().__name__

nombre = "Joselin Liseth Tzunux Coxaj"
edad = 18
promedio = 65.5
activo = True

print(nombre, type(nombre).__name__)
print(edad, type(edad).__name__)
print(promedio, type(promedio).__name__)
print(activo, type(activo).__name__)
```

```
Joselin Liseth Tzunux Coxaj str
18 int
65.5 float
True bool
```

[21]:
```
# Ejercicio 2b
# Usa asignación múltiple para crear tres variables x, y, z
# con los valores 10, 20 y 30 en una sola línea.
# Luego imprime: "x=10, y=20, z=30" usando un f-string.

x, y, z = 10, 20, 30
print(f"x={x}, y={y}, z={z}")
```

```
x=10, y=20, z=30
```

---

## 1.3 3. Operaciones aritméticas

Python tiene los operadores: `+`, `-`, `*`, `/` (división real), `//` (división entera), `%` (módulo), `**` (potencia).
El módulo `math` ofrece funciones como `sqrt`, `ceil`, `floor`.

[24]:
```
# Ejercicio 3a
# Dados a=23 y b=7, calcula e imprime:
#   - La división entera de a entre b
#   - El residuo (módulo) de a entre b
#   - a elevado a la potencia b (solo los primeros dígitos, usa f-string)
```

```
a, b = 23, 7

division_entera = a // b
residuo = a % b
potencia = a ** b

print(f"{a} // {b}  = {3}")
print(f"{a} % {b}  = {2}")
print(f"{a} ** {b}  = {3404825447}")
```

```
23 // 7  = 3
23 % 7  = 2
23 ** 7  = 3404825447
```

[27]:
```
# Ejercicio 3b
# Usa el módulo math para calcular:
#   1. La raíz cuadrada de 144
#   2. El techo (ceil) de 7.3
#   3. El piso (floor) de 7.9
#   4. El valor de pi redondeado a 4 decimales (usa :.4f en el f-string)

import math

raiz = math.sqrt(144)
techo = math.ceil(7.3)
piso = math.floor(7.9)
valor_pi = math.pi

print(f"sqrt(144)   = {math.sqrt(144)}")
print(f"ceil(7.3)   = {math.ceil(7.3)}")
print(f"floor(7.9)  = {math.floor(7.9)}")
print(f"pi          = {math.pi:.4f}")
```

```
sqrt(144)   = 12.0
ceil(7.3)   = 8
floor(7.9)  = 7
pi          = 3.1416
```

---

## 1.4 4. Strings y métodos útiles

Los strings tienen métodos como `.strip()`, `.upper()`, `.lower()`, `.replace()`, `.split()`, `.join()`. Los **f-strings** permiten insertar variables directamente en el texto: `f"Hola {nombre}"`.

[1]:
```
# Ejercicio 4a
# Dada la cadena: "  fisica y matematica  "
# Aplica los métodos para:
```

```python
#   1. Quitar espacios extremos
#   2. Convertir a mayúsculas
#   3. Reemplazar "Y" por "&"
# Imprime el resultado de cada paso.

cadena = "  fisica y matematica  "
paso1 = cadena.strip()
print(f"Paso 1 (strip): '{paso1}'")

paso2 = paso1.upper()
print(f"Paso 2 (upper): {paso2}")

paso3 = paso2.replace("Y", "&")
print(f"Paso 3 (replace): {paso3}")
```

```
Paso 1 (strip): 'fisica y matematica'
Paso 2 (upper): FISICA Y MATEMATICA
Paso 3 (replace): FISICA & MATEMATICA
```

[2]:
```python
# Ejercicio 4b
# Dada la cadena: "rojo-verde-azul-amarillo"
# 1. Sepárala en una lista de colores usando split()
# 2. Une la lista con " | " usando join()
# 3. Verifica con 'in' si "verde" está en la lista (imprime True/False)

colores_str = "rojo-verde-azul-amarillo"

lista_colores = colores_str.split("-")
print(f"Lista: {lista_colores}")

cadena_unida = " | ".join(lista_colores)
print(f"Unida: {cadena_unida}")

esta_verde = "verde" in lista_colores
print(f"¿Está verde?: {esta_verde}")
```

```
Lista: ['rojo', 'verde', 'azul', 'amarillo']
Unida: rojo | verde | azul | amarillo
¿Está verde?: True
```

---

## 1.5 5. Listas

Las listas son **mutables** y se crean con `[]`. Métodos principales: `.append()`, `.insert()`, `.remove()`, `.pop()`.
Funciones útiles: `len()`, `min()`, `max()`, `sum()`, `sorted()`.

[4]:
```python
# Ejercicio 5a
# Crea una lista con las materias: "Cálculo", "Física", "Programación", "Álgebra"
# Luego:
#   1. Agrega "Estadística" al final
#   2. Inserta "Química" en la posición 2
#   3. Elimina "Álgebra" de la lista
#   4. Imprime la lista final y su longitud

materias = ["Cálculo", "Física", "Programación", "Álgebra"]

materias.append("Estadística")

materias.insert(2, "Química")

materias.remove("Álgebra")

print(f"Lista final: {materias}")
print(f"Longitud de la lista: {len(materias)}")
```

```
Lista final: ['Cálculo', 'Física', 'Química', 'Programación', 'Estadística']
Longitud de la lista: 5
```

[42]:
```python
# Ejercicio 5b
# Dada la lista de notas, imprime: mínimo, máximo, suma y promedio.
# El promedio debe mostrarse con 2 decimales usando f-string :.2f

notas = [78, 92, 65, 88, 74, 95, 61, 83]

print(f"Mínimo    : {min(notas)}")
print(f"Máxima    : {max(notas)}")
print(f"Suma      : {sum(notas)}")
print(f"Promedio  : {sum(notas)/len(notas):.2f}")
```

```
Mínimo    : 61
Máxima    : 95
Suma      : 636
Promedio  : 79.50
```

---

## 1.6 6. Diccionarios

Los diccionarios almacenan pares **clave** → **valor**. Se crean con `{}`.
Para acceder de forma segura usa `.get(clave, valor_por_defecto)`.
Para iterar sobre pares: `for clave, valor in d.items():`

[5]:
```python
# Ejercicio 6a
# Crea un diccionario que represente un curso con las claves:
#   "nombre", "codigo", "creditos", "activo"
```

```python
# Luego:
#   1. Agrega la clave "estudiantes" con el valor 35
#   2. Imprime el valor de "creditos" usando .get()
#   3. Intenta obtener la clave "salon" con valor por defecto "Sin asignar"

curso = {
    "nombre": "Programacion",
    "codigo": "101",
    "creditos": 5,
    "activo": True
}

curso["estudiantes"] = 35
print(f"Créditos del curso: {curso.get('creditos')}")
salon = curso.get("salon", "Sin asignar")
print(f"Salón: {salon}")
```

```
Créditos del curso: 5
Salón: Sin asignar
```

[6]:
```python
# Ejercicio 6b
# Itera sobre el diccionario del ejercicio anterior e imprime
# cada clave y su valor en el formato:  clave     : valor
# (usa f-string con alineación, ej: f"{clave:12} : {valor}")

for clave, valor in curso.items():
    print(f"{clave:12} : {valor}")
```

```
nombre       : Programacion
codigo       : 101
creditos     : 5
activo       : True
estudiantes  : 35
```

---

## 1.7 7. Condicionales

Estructura: `if condición:` / `elif condición:` / `else:`
Operador ternario (en una línea): `valor_si_true if condición else valor_si_false`

[7]:
```python
# Ejercicio 7a
# Escribe una función llamada clasificar_imc(imc) que reciba el
# Índice de Masa Corporal y retorne una cadena según esta escala:
#   < 18.5  → "Bajo peso"
#   18.5-24.9 → "Normal"
#   25-29.9 → "Sobrepeso"
#   >= 30   → "Obesidad"
#
```

```python
# Pruébala con los valores: 16.0, 22.5, 27.3, 31.8

def clasificar_imc(imc):
    if imc < 18.5:
        return "Bajo peso"
    elif 18.5 <= imc <= 24.9:
        return "Normal"
    elif 25 <= imc <= 29.9:
        return "Sobrepeso"
    else:
        return "Obesidad"

valores = [16.0, 22.5, 27.3, 31.8]
for v in valores:
    resultado = clasificar_imc(v)
    print(f"IMC {v}: {resultado}")
```

```
IMC 16.0: Bajo peso
IMC 22.5: Normal
IMC 27.3: Sobrepeso
IMC 31.8: Obesidad
```

[50]:
```python
# Ejercicio 7b
# Usa el operador ternario para determinar si un número es par o impar.
# Pruébalo con los números 4, 7 y 0.
# Salida esperada:  4 es par  /  7 es impar  /  0 es par

# Ejercicio 7b
numeros = [4, 7, 0]

for n in numeros:
    # Estructura: valor_si_true if condicion else valor_si_false
    resultado = "par" if n % 2 == 0 else "impar"
    print(f"{n} es {resultado}")
```

```
4 es par
7 es impar
0 es par
```

---

## 1.8 8. Ciclos

- `for elemento in iterable:` — recorre cada elemento
- `for i in range(inicio, fin, paso):` — secuencia numérica
- `for i, val in enumerate(lista):` — índice + valor
- `while condición:` — repite mientras la condición sea verdadera
- **List comprehension:** `[expresión for x in iterable if condición]`

[52]:
```python
# Ejercicio 8a
# Usa un for con enumerate() para imprimir la siguiente lista de planetas
# numerados desde 1.
# Salida esperada:
#   [1] Mercurio
#   [2] Venus
#   ...

planetas = ["Mercurio", "Venus", "Tierra", "Marte", "Júpiter", "Saturno",␣
↪"Urano", "Neptuno"]

for i, planeta in enumerate(planetas, start=1):
    print(f"[{i}] {planeta}")
```

```
[1] Mercurio
[2] Venus
[3] Tierra
[4] Marte
[5] Júpiter
[6] Saturno
[7] Urano
[8] Neptuno
```

[53]:
```python
# Ejercicio 8b
# Usa un ciclo while para imprimir la tabla de multiplicar del 7
# (del 7x1 hasta el 7x10).
# Formato:  7 x 1 = 7

i = 1
while i <= 10:
    print(f"7 x {i} = {7 * i}")
    i += 1
```

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

[54]:
```python
# Ejercicio 8c - List comprehension
# En una sola línea, crea las siguientes listas y luego imprímelas:
#   1. Los cubos de los números del 1 al 8  (x³)
#   2. Los números del 1 al 30 que sean divisibles por 3 pero NO por 9
```

```python
# 1. Los cubos de los números del 1 al 8 (x^3)
cubos = [x**3 for x in range(1, 9)]
print(f"Cubos: {cubos}")

# 2. Números del 1 al 30 divisibles por 3 pero NO por 9
# Usamos el operador % para el residuo
filtrados = [n for n in range(1, 31) if n % 3 == 0 and n % 9 != 0]
print(f"Filtrados: {filtrados}")
```

```
Cubos: [1, 8, 27, 64, 125, 216, 343, 512]
Filtrados: [3, 6, 12, 15, 21, 24, 30]
```

---

## 1.9 9. Funciones

```python
def nombre_funcion(param1, param2=valor_defecto):
    """Docstring: descripción breve."""
    return resultado
```

Una función puede retornar **múltiples valores** (como tupla) y llamarse a sí misma (**recursión**).

[55]:
```python
# Ejercicio 9a
# Escribe una función convertir_temperatura(grados, escala="C") que convierta:
#   - Si escala="C"  → convierte Celsius a Fahrenheit: F = C * 9/5 + 32
#   - Si escala="F"  → convierte Fahrenheit a Celsius: C = (F - 32) * 5/9
#
# Pruébala con: 100°C, 0°C, 32°F y 212°F

def convertir_temperatura(grados, escala="C"):
    if escala == "C":
        # Celsius a Fahrenheit
        return (grados * 9/5) + 32
    elif escala == "F":
        # Fahrenheit a Celsius
        return (grados - 32) * 5/9

# Pruebas
print(f"100°C a F: {convertir_temperatura(100, 'C')}°F")
print(f"0°C a F: {convertir_temperatura(0, 'C')}°F")
print(f"32°F a C: {convertir_temperatura(32, 'F')}°C")
print(f"212°F a C: {convertir_temperatura(212, 'F')}°C")
```

```
100°C a F: 212.0°F
0°C a F: 32.0°F
32°F a C: 0.0°C
212°F a C: 100.0°C
```

[9]:
```python
# Ejercicio 9b
# Escribe una función resumen_lista(lista) que retorne
# tres valores: (minimo, maximo, promedio).
# Úsala con: [12, 45, 7, 89, 34, 56, 23]
# Imprime los tres resultados usando desempaquetado.

def resumen_lista(lista):
    minimo = min(lista)
    maximo = max(lista)
    promedio = sum(lista) / len(lista)
    return minimo, maximo, promedio

# Datos de prueba
numeros = [12, 45, 7, 89, 34, 56, 23]

mi_min, mi_max, mi_prom = resumen_lista(numeros)

print(f"Mínimo: {mi_min}, Máximo: {mi_max}, Promedio: {mi_prom:.2f}")
```

```
Mínimo: 7, Máximo: 89, Promedio: 38.00
```

[10]:
```python
# Ejercicio 9c - Recursión
# Escribe una función recursiva fibonacci(n) que retorne
# el n-ésimo número de la sucesión de Fibonacci.
# Casos base: fibonacci(0)=0, fibonacci(1)=1
# Caso recursivo: fibonacci(n) = fibonacci(n-1) + fibonacci(n-2)
#
# Imprime los primeros 10 números: fibonacci(0) ... fibonacci(9)

def fibonacci(n):
    # Casos base
    if n == 0:
        return 0
    elif n == 1:
        return 1
    # Caso recursivo
    else:
        return fibonacci(n-1) + fibonacci(n-2)

# Imprimir los primeros 10 números
print("Sucesión de Fibonacci:")
for i in range(10):
    print(f"fibonacci({i}) = {fibonacci(i)}")
```

```
Sucesión de Fibonacci:
fibonacci(0) = 0
fibonacci(1) = 1
fibonacci(2) = 1
```

```
fibonacci(3) = 2
fibonacci(4) = 3
fibonacci(5) = 5
fibonacci(6) = 8
fibonacci(7) = 13
fibonacci(8) = 21
fibonacci(9) = 34
```

---

### 1.10 10. Manejo de errores

```python
try:
    # código que puede fallar
except TipoDeError as e:
    # qué hacer si ocurre el error
finally:
    # se ejecuta siempre
```

Para lanzar un error explícitamente: `raise ValueError("mensaje")`

[11]:
```python
# Ejercicio 10a
# Escribe una función convertir_entero(valor) que intente convertir
# el parámetro a entero usando int().
#   - Si tiene éxito, retorna el entero.
#   - Si ocurre un ValueError, imprime un mensaje de error y retorna None.
#   - El bloque finally debe imprimir "[intento con: <valor>]" siempre.
#
# Pruébala con: "42", "3.14", "hola", 100

def convertir_entero(valor):
    try:
        resultado = int(valor)
        return resultado
    except ValueError:
        print(f"Error: No se pudo convertir '{valor}' a entero.")
        return None
    finally:
        print(f"[Intento con: {valor}]")

# Pruebas
pruebas_10a = ["42", "3.14", "hola", 100]
for p in pruebas_10a:
    res = convertir_entero(p)
    print(f"Resultado: {res}\n")
```

```
[Intento con: 42]
Resultado: 42
```

```
Error: No se pudo convertir '3.14' a entero.
[Intento con: 3.14]
Resultado: None

Error: No se pudo convertir 'hola' a entero.
[Intento con: hola]
Resultado: None

[Intento con: 100]
Resultado: 100
```

[12]:
```python
# Ejercicio 10b
# Escribe una función validar_edad(edad) que:
#   - Lance un TypeError  si edad no es int
#   - Lance un ValueError si edad < 0 o edad > 150
#   - Retorne True si la edad es válida
#
# Pruébala dentro de un try/except con: 25, -5, 200, "veinte"

def validar_edad(edad):
    if not isinstance(edad, int):
        raise TypeError("La edad debe ser un número entero.")
    if edad < 0 or edad > 150:
        raise ValueError("La edad debe estar entre 0 y 150.")
    return True

# Pruebas dentro de un try/except
datos_prueba = [25, -5, 200, "veinte"]

for e in datos_prueba:
    try:
        if validar_edad(e):
            print(f"Edad {e} validada correctamente.")
    except (TypeError, ValueError) as error:
        print(f"Error con {e}: {error}")
```

```
Edad 25 validada correctamente.
Error con -5: La edad debe estar entre 0 y 150.
Error con 200: La edad debe estar entre 0 y 150.
Error con veinte: La edad debe ser un número entero.
```

---

## 1.11 Ejercicio integrador

Combina lo aprendido en un mini-programa completo.

[14]:
```python
# Ejercicio integrador
#
# Crea una función analizar_curso(estudiantes) que reciba una lista de
# diccionarios. Cada diccionario representa un estudiante con las claves:
#   "nombre" (str), "notas" (lista de ints)
#
# La función debe:
#   1. Calcular el promedio de cada estudiante (retorna None si notas está vacía)
#   2. Clasificar al estudiante: promedio >= 61 → "Aprobado", si no → "Reprobado"
#   3. Imprimir una tabla con: nombre, promedio (2 decimales) y estado
#   4. Al final, imprimir cuántos aprobaron y cuántos reprobaron
#
# Usa el siguiente listado de prueba:

curso = [
    {"nombre": "Ana López",    "notas": [85, 90, 78, 92]},
    {"nombre": "Carlos Ruiz",  "notas": [55, 48, 60, 52]},
    {"nombre": "María Paz",    "notas": [70, 75, 68, 80]},
    {"nombre": "Luis Gómez",   "notas": [95, 98, 100, 92]},
    {"nombre": "Sara Díaz",    "notas": [40, 55, 50, 45]},
]

def analizar_curso(estudiantes):
    aprobados = 0
    reprobados = 0

    print(f"{'Nombre':<15} | {'Promedio':<10} | {'Estado':<10}")
    print("-" * 40)

    for estudiante in estudiantes:
        nombre = estudiante["nombre"]
        notas = estudiante["notas"]

        # 1. Calcular promedio (retorna None si está vacía)
        if len(notas) == 0:
            promedio = None
            estado = "N/A"
        else:
            promedio = sum(notas) / len(notas)

            # 2. Clasificar al estudiante
            if promedio >= 61:
                estado = "Aprobado"
                aprobados += 1
            else:
                estado = "Reprobado"
                reprobados += 1
```

```python
        # 3. Imprimir tabla formateada
        prom_str = f"{promedio:.2f}" if promedio is not None else "Sin notas"
        print(f"{nombre:<15} | {prom_str:<10} | {estado:<10}")

    # 4. Resumen final
    print("-" * 40)
    print(f"Total Aprobados: {aprobados}")
    print(f"Total Reprobados: {reprobados}")

# Listado de prueba proporcionado
curso = [
    {"nombre": "Ana López", "notas": [85, 90, 78, 92]},
    {"nombre": "Carlos Ruiz", "notas": [55, 48, 60, 52]},
    {"nombre": "Maria Paz", "notas": [70, 75, 88, 80]},
    {"nombre": "Luis Gómez", "notas": [95, 98, 100, 92]},
    {"nombre": "Sara Díaz", "notas": [40, 55, 50, 45]},
]

# Ejecutar la función
analizar_curso(curso)
```

```
Nombre          | Promedio   | Estado
----------------------------------------
Ana López       | 86.25      | Aprobado
Carlos Ruiz     | 53.75      | Reprobado
Maria Paz       | 78.25      | Aprobado
Luis Gómez      | 96.25      | Aprobado
Sara Díaz       | 47.50      | Reprobado
----------------------------------------
Total Aprobados: 3
Total Reprobados: 2
```